05 — Checkpoints

> ***Module:*** *PostgreSQL Internals* | ***Difficulty:*** *Advanced* | ***Est. reading time:*** *45 min*

Table of Contents

Learning Objectives

After completing this section you will be able to:

- Define a checkpoint and explain why it bounds crash recovery time.
- List every action PostgreSQL takes during a checkpoint in order.
- Explain how `checkpoint_completion_target` spreads dirty-page I/O and reduces I/O spikes.
- Distinguish the bgwriter's role from the checkpointer's role.
- Tune checkpoint-related parameters for OLTP and bulk-load workloads.
- Interpret `pg_stat_bgwriter` output to detect I/O-related performance problems.
- Explain the interaction between checkpoints and full-page writes.

What Is a Checkpoint?

A **checkpoint** is a moment in time at which PostgreSQL guarantees that all dirty data pages that were modified before the checkpoint's **REDO LSN** have been flushed from shared_buffers to the data files on disk.

After a checkpoint completes:

- The WAL records before the REDO LSN are no longer needed for crash recovery of data files already on disk.
- Recovery after a crash need only replay WAL from the checkpoint's REDO LSN forward — not from the beginning of WAL.

A checkpoint does NOT:

- Flush WAL itself (WAL is already flushed at each commit).
- Lock out all transactions.
- Guarantee anything about WAL segments — WAL segments older than the checkpoint may still be needed for PITR or replication.

Why Checkpoints Limit Recovery Time

```
WAL Stream:
  LSN 0   LSN A    LSN B             LSN C (crash)
  |       |        |                 |
  |       Ckpt 1   Ckpt 2            crash
  |                |                 |
  |           Recovery after crash at C:
  |           Must replay from LSN B (Ckpt 2 REDO LSN) → much less work
```

Without checkpoints, crash recovery would need to replay WAL from the very beginning of WAL history (or the base backup). With checkpoints every `checkpoint_timeout` (default 5 minutes), recovery replays at most 5 minutes of WAL.

The recovery time is bounded by: `checkpoint_timeout × transaction rate × WAL record size`.

What Happens During a Checkpoint

The **checkpointer** process executes a checkpoint in the following sequence:

```
Step 1: Determine the REDO LSN
  Set redo_lsn = current WAL insert position.
  After this point, any WAL record before redo_lsn is sufficient to recover
  all pages that will be flushed during this checkpoint.

Step 2: Write a "checkpoint in progress" WAL record
  Records the redo_lsn so recovery knows where to start replaying.

Step 3: Flush dirty shared_buffers pages to disk
  - Iterate through all dirty pages in the buffer pool.
  - Write each dirty page to its data file.
  - Pages are written in order of their on-disk location (to minimise seek time).
  - Spread over checkpoint_completion_target × checkpoint_interval time.

Step 4: fsync all modified data files
  Ensures OS page cache has truly written data to persistent storage.

Step 5: Update the control file
  pg_control is updated with the new checkpoint LSN and timestamp.
  This is the authoritative record of the last successful checkpoint.

Step 6: Recycle old WAL segments
  WAL segments before redo_lsn that are not needed for replication or
  archiving may now be recycled.
```

The checkpoint process is designed to be I/O-smooth — it does not lock tables and spreads its writes across the checkpoint interval.

---

checkpoint_completion_target: Spreading I/O

Without throttling, the checkpointer would flush all dirty pages as fast as possible — causing a burst of write I/O that starves other operations. `checkpoint_completion_target` tells the checkpointer what fraction of the checkpoint interval to use for flushing.

```
checkpoint_timeout = 5 min
checkpoint_completion_target = 0.9

Target: finish flushing dirty pages within 0.9 × 5 min = 4.5 min
Remaining 0.5 min: fsync + WAL record + control file update

Effect: checkpointer throttles its write rate to spread 4.5 min of I/O
        evenly, rather than a sudden burst.
```

The default `0.9` is appropriate for most workloads. Lower values (e.g., `0.5`) complete checkpoints faster but create more I/O spikes.

The Background Writer (bgwriter)

The **bgwriter** is a separate process from the checkpointer. Its job is to proactively write "clean" dirty pages from shared_buffers to disk *between* checkpoints, so that when a checkpoint fires there are fewer dirty pages to flush.

Key parameters:

Parameter	Default	Effect
`bgwriter_delay`	200 ms	Sleep time between bgwriter rounds
`bgwriter_lru_maxpages`	100	Max pages written per round
`bgwriter_lru_multiplier`	2.0	Multiple of recently evicted pages to write proactively
`bgwriter_flush_after`	512 kB	Flush to OS after this many bytes written

The bgwriter also handles buffer eviction: when a backend needs a new buffer slot and shared_buffers is full, the backend may need to write a dirty page itself (a **backend write**). A high `buffers_backend` count in `pg_stat_bgwriter` indicates the bgwriter is not keeping up.

---

checkpoint_timeout vs max_wal_size

A checkpoint is triggered by **whichever limit is hit first**:

Trigger	Parameter	Default	When it fires
Time-based	`checkpoint_timeout`	5 min	Every 5 minutes (timed checkpoint)

WAL-size-based	`max_wal_size`	1 GB	When WAL has grown by 1 GB since last checkpoint

```
checkpoint_timeout = 5min, max_wal_size = 1GB

Scenario A: Low write rate → time triggers checkpoint (timed)
  → logs "LOG: checkpoint complete: ... reason: time"

Scenario B: High write rate → WAL grows 1 GB in 2 minutes → WAL triggers checkpoint
  → logs "LOG: checkpoint complete: ... reason: wal"
  → If this happens frequently, increase max_wal_size
```

`min_wal_size` (default 80 MB) controls the minimum number of WAL segments kept around so they can be recycled, avoiding filesystem allocation overhead.

Forced Checkpoints

Checkpoints can also be triggered outside the normal schedule:

Cause	Notes
`CHECKPOINT` command	Manual; can be run by superuser. Completes synchronously.
`pg_basebackup` start	A checkpoint at start of base backup ensures a consistent starting point.
`CREATE DATABASE` / `DROP DATABASE`	Requires a checkpoint to safely complete.
Server shutdown	A final checkpoint flushes all dirty pages before exit.
Before/after recovery	A checkpoint is written at the end of crash recovery.

Frequent forced checkpoints (especially from `CREATE/DROP DATABASE`) can spike I/O. Monitor for unexpected checkpoint reasons in the PostgreSQL log.

The WAL Checkpoint Record

When a checkpoint fires, PostgreSQL writes a special WAL record of type `XLOG_CHECKPOINT_ONLINE` (during a running system) or `XLOG_CHECKPOINT_SHUTDOWN` (during clean shutdown):

```c
typedef struct CheckPoint {
    XLogRecPtr  redo;           /* REDO LSN: replay from here on recovery */
    TimeLineID  ThisTimeLineID;
    TimeLineID  PrevTimeLineID; /* previous timeline (for PITRed standbys) */
    bool        fullPageWrites;
    uint64      nextXid;        /* next available transaction ID */
    Oid         nextOid;        /* next available OID */
    MultiXactId nextMulti;
    MultiXactOffset nextMultiOffset;
    TransactionId oldestXid;
```

```
    Oid         oldestXidDB;
    TransactionId oldestActiveXid;
    /* ... more fields ... */
    pg_time_t    time;          /* time of checkpoint */
} CheckPoint;
```

The `redo` field is what recovery uses as the starting point. `pg_controldata` shows the last checkpoint's values.

```
pg_controldata $PGDATA | grep -i checkpoint
```

Checkpoint and Full-Page Writes

After each checkpoint, the **first write** to any heap or index page causes a full-page image (FPI) to be written to WAL. This is reset at the next checkpoint.

Impact: Frequent checkpoints → more FPI events → more WAL volume, but each individual checkpoint has fewer dirty pages to flush.

Trade-off:

- Large `checkpoint_timeout` (30 min) → fewer FPIs, less WAL, but longer recovery time.
- Small `checkpoint_timeout` (1 min) → more FPIs, more WAL, but faster recovery.

Recommendation: Keep `checkpoint_timeout` at 5–30 minutes depending on acceptable recovery time. Use `max_wal_size` to cap WAL retention.

ASCII Diagrams

Checkpoint Timeline

```
Time ──────────────────────────────────────────────────────────────────►

Ckpt1                    Ckpt2                    Ckpt3
  │                        │                        │
  │◄─── 5 min ────────────►│◄─── 5 min ────────────►│
  │                        │                        │
  │  checkpoint_            │  checkpoint_            │
  │  completion_            │  completion_            │
  │  target=0.9:            │  target=0.9:            │
  │  4.5 min writes         │  4.5 min writes         │
  │  0.5 min fsync          │  0.5 min fsync          │
```

Dirty Page Flush Rate

```
Dirty pages
in shared_buffers
    │
    │ ██████████ (burst at checkpoint without completion_target)
    │ vs.
```

```
    │ ░░░░░░░░░░░░░░░░░░░░░░░░░░░░░░░░░░░░░░░ (spread with 0.9)
    │
    └─────────────────────────────────────────────▶ time
         Ckpt start                       Ckpt end
```

bgwriter vs Checkpointer

```
                    shared_buffers
                   ┌─────────────────┐
INSERTS ───────▶   │ dirty pages     │
UPDATES ───────▶   │                 │
                   └───────┬─────────┘
                           │
         ┌─────────────────┴──────────────────┐
         │                                    │
         ▼                                    ▼
bgwriter (proactive)              checkpointer (periodic)
- writes dirty pages              - flushes ALL dirty pages
  before eviction                   before redo LSN
- reduces checkpoint work         - fsyncs data files
- driven by bgwriter_lru_*        - updates pg_control
parameters                       - driven by timeout/wal_size
```

Monitoring Checkpoints

```
-- pg_stat_bgwriter: the primary checkpoint monitoring view
SELECT
    checkpoints_timed,      -- triggered by checkpoint_timeout
    checkpoints_req,        -- triggered by max_wal_size or manual CHECKPOINT
    checkpoint_write_time,  -- ms spent writing dirty pages
    checkpoint_sync_time,   -- ms spent in fsync
    buffers_checkpoint,     -- pages written by checkpointer
    buffers_clean,          -- pages written by bgwriter (proactive)
    maxwritten_clean,       -- times bgwriter hit bgwriter_lru_maxpages limit
    buffers_backend,        -- pages written by backends (bad! bgwriter not keeping
up)
    buffers_backend_fsync,  -- times backend had to fsync (very bad)
    buffers_alloc,          -- total buffer allocations
    stats_reset
FROM pg_stat_bgwriter;

-- Derived metrics
SELECT
    checkpoints_timed + checkpoints_req AS total_checkpoints,
    CASE checkpoints_timed + checkpoints_req
        WHEN 0 THEN NULL
        ELSE round(100.0 * checkpoints_req / (checkpoints_timed + checkpoints_req),
1)
    END AS pct_forced,       -- high % means max_wal_size too low
```

```
    round(buffers_checkpoint::numeric /
          NULLIF(checkpoints_timed + checkpoints_req, 0), 0) AS
avg_buffers_per_checkpoint,
    buffers_backend          -- >0 = bgwriter can't keep up
FROM pg_stat_bgwriter;

-- Check checkpoint log messages
-- In postgresql.conf: log_checkpoints = on
-- Then: SELECT * FROM pg_catalog.pg_available_extensions WHERE name =
'pg_log_userqueries';
```

Common Mistakes

1. **`checkpoints_req >> checkpoints_timed`** — forced checkpoints from WAL size. Solution: increase `max_wal_size`.
2. **High `buffers_backend`** — backends are writing dirty pages because bgwriter can't keep up. Tune `bgwriter_lru_maxpages` and `bgwriter_lru_multiplier`.
3. **High `checkpoint_sync_time`** — fsync is slow; indicates storage subsystem bottleneck or insufficient I/O bandwidth. Consider dedicated storage for WAL.
4. **Setting `checkpoint_timeout = 1min` on OLTP** — too frequent checkpoints increase full-page write volume and WAL size.
5. **Setting `max_wal_size` too small** — forces checkpoints before `checkpoint_timeout`, leading to excessive checkpoint frequency.
6. **Forgetting `log_checkpoints = on`** — without this, you have no visibility into checkpoint frequency or timing in the PostgreSQL log.
7. **Running `CHECKPOINT` manually during peak hours** — a manual checkpoint is not throttled by `checkpoint_completion_target` and will use maximum I/O bandwidth.

Best Practices

- **Enable `log_checkpoints = on`** in production to track checkpoint frequency.
- **Set `max_wal_size` to 2–8 GB** on busy servers to prevent too-frequent checkpoints.
- **Keep `checkpoint_completion_target = 0.9`** to spread I/O.
- **Tune bgwriter** if `buffers_backend > 0` is persistent: increase `bgwriter_lru_maxpages`.
- **Measure `checkpoint_sync_time`** — if it is consistently > 1 second, the storage subsystem needs attention.
- **Before a large bulk load**, run a manual `CHECKPOINT` to start from a clean state, reducing checkpoint pressure during the load.
- **Set `checkpoint_timeout = 30min`** for data warehouse / OLAP workloads with low transaction rates.

Performance Considerations

Scenario	Symptom	Tuning

Frequent forced checkpoints	`checkpoints_req` high	Increase `max_wal_size`
I/O spikes every 5 minutes	`checkpoint_write_time` spikes	Increase `checkpoint_timeout`, `checkpoint_completion_target = 0.9`
Slow fsync	`checkpoint_sync_time` > 1s	Faster storage, dedicated WAL disk
Backend writes (bgwriter lag)	`buffers_backend` > 0	Increase `bgwriter_lru_maxpages`
Large recovery time	Crash recovery takes minutes	Decrease `checkpoint_timeout`
High WAL volume from FPW	WAL 3–5× expected size	Increase `checkpoint_timeout` to reduce FPW frequency

---

Interview Questions

Q1. What is a checkpoint and why does PostgreSQL need them?

> *A checkpoint is a point in time at which all dirty data pages modified before the checkpoint's REDO LSN are guaranteed to be on disk. They bound crash recovery time: after a crash, PostgreSQL only needs to replay WAL from the last checkpoint's REDO LSN forward, not from the beginning of WAL history.*

Q2. What is the difference between the bgwriter and the checkpointer?

> *The bgwriter proactively writes dirty pages from shared_buffers to disk between checkpoints to reduce the number of dirty pages that need to be flushed at checkpoint time. The checkpointer runs at checkpoint time, flushes all dirty pages modified before the REDO LSN, fsyncs the data files, and updates the control file. They are separate processes.*

Q3. What does `checkpoint_completion_target = 0.9` mean?

> *The checkpointer spreads its dirty-page writes over 90% of the checkpoint interval (by default 4.5 minutes of a 5-minute interval), so I/O is steady rather than a spike. The remaining 10% is reserved for fsync and the checkpoint WAL record.*

Q4. What triggers a checkpoint besides `checkpoint_timeout`?

> `max_wal_size` *(WAL has grown too large since the last checkpoint), manual* `CHECKPOINT` *command, server shutdown,* `pg_basebackup` *start,* `CREATE DATABASE`*,* `DROP DATABASE`*, and end of crash recovery.*

Q5. What does the `pg_control` file record?

> *The* `$PGDATA/global/pg_control` *file stores the last completed checkpoint's LSN, REDO LSN, timeline ID, next transaction ID, next OID, full-page-writes setting, and other critical cluster state. At startup, PostgreSQL reads this file to determine where to start recovery.*

Q6. Why does `buffers_backend > 0` in `pg_stat_bgwriter` indicate a problem?

> `buffers_backend` *counts times a backend process had to write a dirty page to disk because it needed a buffer slot and found no clean pages available. This means the bgwriter and checkpointer are not keeping up with the write rate, forcing backends into I/O work they should not be doing. This increases query latency.*

Q7. How do you detect if checkpoints are happening too frequently?

> *Check `checkpoints_req / (checkpoints_timed + checkpoints_req)`. A high percentage of required (forced) checkpoints means `max_wal_size` is too low for the write rate. Also enable `log_checkpoints = on` to see timing in the PostgreSQL log.*

Q8. What is the relationship between checkpoint frequency and WAL full-page write volume?

> *After each checkpoint, the first modification to any page writes a full-page image (8 KB) to WAL. More frequent checkpoints → more "first modifications after checkpoint" → more FPW WAL records. Reducing checkpoint frequency (larger `checkpoint_timeout`) reduces WAL volume from FPW.*

Exercises with Solutions

Exercise 1 — Checkpoint monitoring dashboard

```
-- Solution: comprehensive checkpoint health view
SELECT
    checkpoints_timed,
    checkpoints_req,
    round(100.0 * checkpoints_req /
          NULLIF(checkpoints_timed + checkpoints_req, 0), 1) AS pct_forced,
    round(checkpoint_write_time / 1000.0, 2)  AS write_time_sec,
    round(checkpoint_sync_time  / 1000.0, 2)  AS sync_time_sec,
    buffers_checkpoint,
    buffers_clean,
    buffers_backend,
    maxwritten_clean,
    CASE
        WHEN buffers_backend > 0 THEN 'WARNING: backends doing I/O'
        WHEN maxwritten_clean > 0 THEN 'INFO: bgwriter hitting limit'
        WHEN checkpoints_req > checkpoints_timed THEN 'WARNING: frequent forced
checkpoints'
        ELSE 'OK'
    END AS health_status
FROM pg_stat_bgwriter;
```

Exercise 2 — Trigger and observe a checkpoint

```
-- Solution
-- 1. Reset stats
SELECT pg_stat_reset_shared('bgwriter');

-- 2. Generate some dirty pages
CREATE TABLE checkpoint_test AS
SELECT i, md5(i::text) AS data FROM generate_series(1, 100000) i;

-- 3. Check dirty state (before checkpoint)
SELECT count(*) FROM pg_buffercache WHERE isdirty AND relfilenode > 0;
-- (requires pg_buffercache extension)
```

```sql
-- 4. Force a checkpoint
CHECKPOINT;

-- 5. Check again (should be much lower)
SELECT count(*) FROM pg_buffercache WHERE isdirty AND relfilenode > 0;

-- 6. Review stats
SELECT checkpoints_timed, checkpoints_req,
       buffers_checkpoint, checkpoint_write_time, checkpoint_sync_time
FROM pg_stat_bgwriter;
```

Exercise 3 — Parameter audit

```sql
-- Solution: show all checkpoint-related parameters
SELECT name, setting, unit, short_desc
FROM pg_settings
WHERE name IN (
    'checkpoint_timeout', 'checkpoint_completion_target',
    'max_wal_size', 'min_wal_size',
    'bgwriter_delay', 'bgwriter_lru_maxpages', 'bgwriter_lru_multiplier',
    'bgwriter_flush_after', 'checkpoint_flush_after',
    'log_checkpoints'
)
ORDER BY name;
```

Cross-References

- **04_wal.md** — WAL records, LSN, full-page writes interaction
- **06_vacuum.md** — VACUUM also generates WAL and dirty pages
- **08_visibility_map.md** — VM bits reset during aggressive vacuum (freeze)
- **01_storage_architecture.md** — Data files that checkpoint flushes to